TCP-CONTROL

TCP 연결·종료·흐름제어·혼잡제어

연관 기출 · 130-3-3, 133-1-13, 137-3-5

한눈에 이해

TCP 송신자는 수신 버퍼에 들어갈 양과 네트워크가 감당할 양을 따로 계산합니다. 전송 중 바이트가 빠지면 다시 보냅니다.

정의와 핵심 원리

TCP는 바이트 스트림을 순서대로 전달하는 연결 지향 프로토콜입니다. 손실된 데이터는 재전송하고 중복 데이터는 제거합니다. 흐름 제어는 수신 버퍼를, 혼잡 제어는 네트워크 경로를 보호합니다.

• SYN → SYN+ACK → ACK 순서로 연결을 맺고 양쪽의 초기 순서 번호를 확인합니다. SYN과 FIN은 순서 번호 하나를 차지합니다. 데이터가 없는 ACK는 순서 번호를 소비하지 않습니다. 두 방향을 독립적으로 닫으므로 종료는 흔히 FIN과 ACK를 네 번 교환하는 예로 설명하지만, FIN과 ACK를 합쳐 보낼 수도 있습니다.

• 송신자는 혼잡 윈도우(cwnd)와 수신 측이 알린 윈도우(rwnd) 중 작은 값으로 전송량을 제한합니다. 이미 보냈으나 확인을 받지 못한 데이터도 이 한도에 포함합니다. ACK를 받으면 슬라이딩 윈도우의 시작점을 앞으로 옮겨 새 데이터를 보낼 여유가 생깁니다.

• RFC 5681의 느린 시작(slow start)은 ACK를 받을 때마다 윈도우를 빠르게 늘립니다. 혼잡 회피 단계에 들어가면 대체로 왕복 시간(RTT)마다 최대 세그먼트 크기(MSS) 하나만큼 늘립니다. 세 번째 중복 ACK로 손실을 추정한 경우와 재전송 타이머가 만료된 경우의 대응은 다릅니다. CUBIC·ECN 등 구현과 신호의 차이도 구분합니다.

구조·비교·판단

구분	핵심 원리	적용·한계
흐름 제어	rwnd로 수신 버퍼 보호	수신기가 알린 여유가 기준
혼잡 제어	cwnd로 경로 과부하 완화	손실·지연·ECN 등의 신호 참고
오류제어	ACK·재전송·순서 번호	신뢰성과 중복 제거

적용 예제

창과 재전송 타이머

강사 생성 학습 예제입니다. 공식 기준답안이 아닙니다.

조건: 새로 보낼 수 있는 양을 계산하는 시점에는 cwnd = 12 MSS, rwnd = 8 MSS, 아직 확인을 받지 못한 데이터는 5 MSS입니다. 재전송 계산에는 RFC 5681의 별도 상황을 가정합니다.

1. 새로 보낼 수 있는 양은 min(12, 8) – 5 = 3 MSS입니다.

2. 별도로, 처음 재전송 타이머가 만료되기 직전 미확인 데이터(FlightSize)가 8 MSS라고 가정합니다. 이때 ssthresh의 상한은 max(8/2, 2) = 4 MSS입니다.

3. 재전송 타이머가 만료되면 cwnd를 최대 1 MSS로 줄이고 느린 시작으로 복구합니다.

4. 중복 ACK가 세 번 도착하면 타이머 만료를 기다리지 않고 빠른 재전송을 할 수 있습니다. 재전송 후에는 빠른 복구 절차를 따릅니다.

결과·해석: 새로 보낼 양은 수신 버퍼와 혼잡 윈도우로 계산합니다. 재전송 후 혼잡 윈도우는 손실을 감지한 방식에 따라 달라집니다.

전제·주의: RFC 5681의 기본 알고리즘을 설명하는 예제입니다. 초기 윈도우와 혼잡 대응은 구현별로 달라질 수 있습니다.

답안 골격과 혼동 방지

연결·종료 순서 → 순서 번호와 ACK → rwnd·cwnd 비교 → 혼잡 제어 알고리즘 네 가지 → 전송 가능량 계산

- TCP가 메시지 경계를 보존한다고 쓰지 않습니다.
- FIN은 즉시 양방향 전체 종료를 뜻하지 않습니다.

내용을 가리고 회상

1. 흐름제어와 혼잡제어의 보호 대상은 무엇입니까?

2. SYN에 대한 ACK 번호가 1 증가하는 이유는 무엇입니까?

3. 타이머 만료 후 재전송과 중복 ACK에 따른 빠른 복구를 왜 구분합니까?

적용 기준과 근거

명시한 출처 판본의 원리를 설명하며 제품별 구현 차이는 별도 확인합니다.

내용 검토 상태: 출처 확인. 출처 확인은 공식 정답이나 학습 성취를 뜻하지 않습니다.

출처별 상세 열람 범위는 위키의 출처 기록에서 확인합니다.

- RFC9293: Transmission Control Protocol - W. Eddy (ed.) / IETF; 2022-08; 확인 2026-11-05.
- RFC5681: TCP Congestion Control - M. Allman; V. Paxson; E. Blanton / IETF; 2009-09; 확인 2026-11-05.